

AGENTPLACE

Marketplace d'Agents IA

CAHIER DES CHARGES COMPLET

Version 2.0 — Mars 2026

Équipe : 2 développeurs full-stack
Durée estimée : 24 semaines (12 sprints de 2 semaines)
Livable : Web + Desktop (Tauri) + Mobile (React Native)

Document confidentiel

Table des Matières

Table des Matières	2
1. Présentation Générale	5
1.1 Vision du Projet	5
1.2 Analyse Concurrentielle	5
1.3 Objectifs Stratégiques	5
1.4 Parties Prenantes	6
1.5 Organisation de l'Équipe	6
2. Périmètre Fonctionnel Détaillé	7
2.1 Authentification & Gestion des Comptes	7
2.1.1 Inscription et Connexion	7
2.1.2 Gestion de Profil	7
2.1.3 Système de Rôles et Permissions	7
2.2 Catalogue & Découverte d'Agents	7
2.2.1 Page d'Accueil	7
2.2.2 Catalogue et Recherche	8
2.2.3 Fiche Détaillée d'un Agent	8
2.2.4 Collections et Favoris	8
2.2.5 Recommandations Personnalisées	9
2.3 Publication et Gestion des Agents	9
2.3.1 Processus de Publication	9
2.3.2 Pipeline de Validation Automatisée	9
2.3.3 Gestion des Versions	10
2.3.4 Dashboard Développeur	10
2.4 Système de Paiements	10
2.4.1 Architecture Stripe Connect	10
2.4.2 Modèles de Tarification	10
2.4.3 Gestion des Crédits (Pay-per-use)	11
2.4.4 Flux de Paiement Détaillé	11
2.4.5 Mode Sandbox (Essai Avant Achat)	11
2.5 Format Standard .agentjson v2	11
2.5.1 Schéma Complet	12
2.5.2 Système de Permissions	12
2.6 Application Desktop (Tauri 2.0)	13
2.6.1 Fonctionnalités Core	13
2.6.2 Fonctionnalités Avancées	13
2.6.3 Plateformes Supportées	14
2.7 Application Mobile (React Native / Expo)	14
2.7.1 Fonctionnalités Core	14
2.7.2 Fonctionnalités Spécifiques Mobile	14

2.7.3 Plateformes Supportées	15
2.8 Orchestration Multi-Agents (Agent Pipelines)	15
2.8.1 Concept	15
2.8.2 Interface de Création	15
2.8.3 Moteur d'Exécution	15
2.9 Agent Sélecteur Intelligent	16
2.9.1 Fonctionnement	16
2.9.2 Implémentation Technique	16
3. Architecture Technique	17
3.1 Vue d'Ensemble	17
3.2 Stack Technologique Détaillé	17
3.3 Modèle de Données	18
3.4 Architecture API	19
4. Contraintes & Exigences Non-Fonctionnelles	20
4.1 Sécurité	20
4.2 Performance	20
4.3 Observabilité et Monitoring	21
4.4 Accessibilité	21
4.5 Légal et Conformité	21
4.6 Disaster Recovery	22
5. Découpage en Sprints	22
Sprint 0 — Fondations & Setup Semaines 1-2	22
Sprint 1 — Authentification & Profils Semaines 3-4	23
Sprint 2 — Catalogue & Découverte Semaines 5-6	23
Sprint 3 — Publication d'Agents Semaines 7-8	24
Sprint 4 — Paiements & Acquisitions Semaines 9-10	24
Sprint 5 — App Desktop — Fondations Semaines 11-12	25
Sprint 6 — App Desktop — Moteur d'Exécution Semaines 13-14	25
Sprint 7 — Avis, Analytics & Recommandations Semaines 15-16	26
Sprint 8 — Application Mobile MVP Semaines 17-18	26
Sprint 9 — Orchestration Multi-Agents Semaines 19-20	27
Sprint 10 — Fonctionnalités Mobile Avancées & Sprint Buffer Semaines 21-22	27
Sprint 11 — Sécurité, Polish & Lancement Semaines 23-24	27
6. Roadmap et Jalons	28
7. Critères de Succès	29
8. Modèle Économique	29
8.1 Sources de Revenus	29
8.2 Coûts d'Opération Mensuels Estimés	30
9. MVP — Stratégie de Lancement à Coût Zéro	31
9.1 Périmètre du MVP	31
9.1.1 Ce qui est INCLUS dans le MVP	31

9.1.2 Ce qui est EXCLU du MVP (reporté au CDC complet).....	31
9.2 Stack Technique MVP (100% Gratuit)	32
9.3 Architecture Simplifiée du MVP	33
9.3.1 Architecture Mono-Deployment	33
9.3.2 Modèle de Données MVP (réduit)	33
9.4 Fonctionnalités MVP Détaillées.....	34
9.4.1 Landing Page.....	34
9.4.2 Catalogue	34
9.4.3 Fiche Agent.....	34
9.4.4 Publication d'Agents (Développeur)	34
9.4.5 Panel Admin	35
9.4.6 Interface de Chat Web (Cœur du MVP)	35
9.5 Format .agentjson MVP (simplifié)	36
9.6 Design et UX du MVP	36
9.7 Limites Acceptées et Risques du MVP	37
9.8 Critères de Succès du MVP	37
9.9 Stratégie de Transition MVP → CDC Complet.....	38
10. Découpage en Sprints MVP.....	39
Sprint MVP-1 — Fondations & Catalogue Semaines 1-2	39
Sprint MVP-2 — Chat, Admin & Lancement Semaines 3-4.....	40
10.1 Roadmap Complète (MVP + CDC)	40
11. Glossaire.....	41

1. Présentation Générale

1.1 Vision du Projet

AgentPlace est la première marketplace ouverte et model-agnostic d'agents IA. La plateforme permet aux développeurs de publier, monétiser et distribuer leurs agents intelligents, tandis que les utilisateurs finaux peuvent les découvrir, tester en sandbox, acheter et orchestrer via des applications web, desktop et mobile.

Le projet repose sur trois piliers différenciateurs : un format ouvert standardisé (.agentjson), un moteur d'exécution hybride permettant l'exécution locale ou cloud, et un système d'orchestration multi-agents unique sur le marché.

1.2 Analyse Concurrentielle

Le marché des marketplaces d'agents IA est en formation rapide. Le positionnement d'AgentPlace doit être clair face aux acteurs existants.

Concurrent	Modèle	Forces	Faiblesses vs AgentPlace
OpenAI GPT Store	Fermé (GPT-4/o uniquement)	Base utilisateurs massive, intégration ChatGPT	Verrouillé sur un seul modèle, pas d'exécution locale, pas de format ouvert
HuggingFace Spaces	Open source	Communauté ML très active, hébergement gratuit	Pas orienté agents, pas de monétisation native, UX technique
LangChain Hub	Développeur-centric	Framework populaire, composants réutilisables	Pas de marketplace B2C, pas d'UI utilisateur final
CrewAI	Multi-agents	Orchestration native entre agents	Framework pur, pas de plateforme de distribution
Relevance AI	No-code agents	Interface visuelle, déploiement rapide	Pas de marketplace communautaire, pricing élevé

Positionnement AgentPlace

Seule plateforme combinant marketplace B2C ouverte, format standardisé, exécution hybride local/cloud, et orchestration multi-agents. Cible principale : démocratiser l'accès aux agents IA pour les non-développeurs tout en offrant aux développeurs un canal de monétisation simple et transparent.

1.3 Objectifs Stratégiques

- **Créer un écosystème ouvert** : Format .agentjson comme standard de référence pour décrire des agents IA, indépendant de tout fournisseur.
- **Monétisation développeur** : Permettre à tout développeur de publier et vendre ses agents en quelques minutes, avec un dashboard analytique complet.
- **Expérience multiplateforme** : Web (Next.js), Desktop (Tauri pour macOS/Windows/Linux), Mobile (React Native pour iOS/Android).

- **Distribution hybride** : Mode Cloud (endpoint API) et mode Local (exécution sur la machine de l'utilisateur avec sa propre clé API).
- **Orchestration multi-agents** : Fonctionnalité phare permettant de chaîner plusieurs agents dans des workflows automatisés.
- **Sécurité by design** : Système de permissions déclaratif, sandboxing, analyse statique automatisée des agents avant publication.

1.4 Parties Prenantes

Rôle	Description	Besoins clés
Développeur (vendeur)	Crée et publie des agents IA sur la marketplace	Upload simple, dashboard revenus/analytics, documentation API, CLI de publication
Utilisateur final	Découvre, teste et utilise les agents	Découverte intuitive, sandbox d'essai, paiement sécurisé, apps fluides
Administrateur	Gère et modère la plateforme	Modération automatisée, gestion utilisateurs, métriques globales, alertes
Entreprise (futur)	Utilise des agents en équipe	Gestion de licences, facturation centralisée, SSO, agents privés, audit log

1.5 Organisation de l'Équipe

Le projet est porté par une équipe de 2 développeurs full-stack travaillant en pair programming et en répartition de responsabilités.

Rôle	Responsabilités principales	Stack principal
Développeur A — Lead Frontend & Apps	Marketplace web (Next.js), App Desktop (Tauri), App Mobile (React Native), UI/UX, intégrations frontend	React, TypeScript, Tauri, Expo, Tailwind CSS, Figma
Développeur B — Lead Backend & Infra	API backend (Fastify), base de données (Supabase), paiements (Stripe), sécurité, CI/CD, monitoring, moteur d'exécution agents	Node.js, PostgreSQL, Redis, Docker, GitHub Actions, Sentry

Méthode de travail

Sprints de 2 semaines avec daily standup (15 min) et retrospective en fin de sprint.

Git Flow : branche main (production), develop (intégration), feature branches avec PR obligatoires et code review croisée.

Pair programming sur les fonctionnalités critiques (paiements, sécurité, moteur d'exécution).

Outils : GitHub (code + issues + projects), Figma (design), Linear ou GitHub Projects (tickets), Discord/Slack (communication).

2. Périmètre Fonctionnel Détaillé

2.1 Authentification & Gestion des Comptes

2.1.1 Inscription et Connexion

- Inscription par email + mot de passe avec validation (longueur min 8 caractères, complexité).
- Connexion OAuth via GitHub, Google et Apple (Supabase Auth).
- Vérification d'email obligatoire avant activation du compte.
- Récupération de mot de passe par email avec lien temporaire (expiration 1h).
- Sélection du type de compte à l'inscription : Utilisateur ou Développeur.
- Possibilité d'upgrader un compte Utilisateur vers Développeur à tout moment.

2.1.2 Gestion de Profil

- Profil utilisateur : avatar, nom d'affichage, bio, préférences de langue, gestion des clés API personnelles.
- Profil développeur public : bio, lien site/GitHub, portfolio d'agents publiés, note moyenne, badge de vérification, nombre total de téléchargements.
- Gestion des clés API : stockage chiffré AES-256 côté serveur. L'utilisateur peut ajouter ses clés pour Claude, GPT-4, Gemini, Mistral, etc. Les clés ne sont jamais exposées côté client (affichage masqué après saisie).
- Gestion des sessions : JWT avec refresh token, expiration access token 1h, refresh token 7 jours. Déconnexion de toutes les sessions depuis les paramètres.

2.1.3 Système de Rôles et Permissions

Rôle	Permissions	Accès
Utilisateur	Parcourir le catalogue, acheter des agents, noter et commenter, gérer sa bibliothèque	Web, Desktop, Mobile
Développeur	Toutes les permissions Utilisateur + publier des agents, accéder au dashboard analytics, gérer les versionnages	Web, Desktop, Mobile
Administrateur	Toutes les permissions + modération des agents, gestion des utilisateurs, accès aux métriques globales, gestion des signalements	Web (panel admin)

2.2 Catalogue & Découverte d'Agents

2.2.1 Page d'Accueil

La page d'accueil de la marketplace présente une vue synthétique et engageante de l'écosystème d'agents disponibles, organisée en sections dynamiques.

- **Hero section** : Barre de recherche prominente, tagline, compteur d'agents disponibles, call-to-action.
- **Agents en vedette** : Sélection éditoriale (manuelle par l'admin) de 4 à 6 agents mis en avant avec visuels larges.
- **Tendances** : Top 10 agents par nombre d'achats/installations sur les 7 derniers jours, calculé via un job CRON quotidien.
- **Nouveautés** : Derniers agents publiés (status approved), triés par date, limité à 12.
- **Catégories** : Grille de catégories avec icônes et compteurs d'agents : Productivité, Dev & Code, Marketing, Finance, Créatif, Data, Santé, Éducation, Autres.

2.2.2 Catalogue et Recherche

- Page catalogue avec grille de cards agents (nom, icône, description courte, prix, note moyenne, nombre d'avis).
- Filtres combinables : catégorie, modèle IA (Claude, GPT-4, Gemini, Mistral, Autre), fourchette de prix (gratuit, < 10€, < 50€, > 50€), note minimum, mode (local/cloud/hybride), tri (popularité, date, prix, note).
- Moteur de recherche full-text via Supabase (ts_vector + ts_query) combiné avec pgvector pour la recherche sémantique. L'utilisateur tape en langage naturel, le système retourne les agents les plus pertinents.
- Pagination avec lazy loading (20 agents par page, chargement au scroll).
- Cache Redis (TTL 5 min) sur les requêtes catalogue les plus fréquentes pour des réponses < 100ms.

2.2.3 Fiche Détaillée d'un Agent

- Nom, icône, description longue (Markdown rendu en HTML), version actuelle, auteur avec lien vers profil.
- Captures d'écran et vidéo de démo (upload par le développeur, hébergé sur Supabase Storage / R2).
- Informations techniques : modèle(s) IA supporté(s), mode (local/cloud/hybride), permissions requises, dépendances.
- Tarification clairement affichée avec bouton d'achat ou bouton « Essayer en Sandbox ».
- Section avis : note moyenne, distribution des notes (graphique barres), liste des commentaires paginés avec filtre (les plus récents, les plus utiles).
- Changelog des versions avec possibilité de rollback pour les propriétaires.
- Boutons d'action : Acheter, Essayer (sandbox), Ajouter aux favoris, Partager (lien avec OG tags).

2.2.4 Collections et Favoris

- Chaque utilisateur peut créer des collections nommées (ex : « Mes agents de productivité ») et y ajouter des agents.
- Les collections peuvent être publiques (partageables par URL) ou privées.
- Bouton « Favori » (cœur) sur chaque card agent pour un accès rapide.

- Page « Mes Favoris » accessible depuis le profil.

2.2.5 Recommandations Personnalisées

- Système de recommandation basé sur pgvector : les agents sont représentés par des embeddings calculés à partir de leur description, tags et catégorie.
- Section « Agents similaires » sur chaque fiche agent (5 agents les plus proches par cosine similarity).
- Section « Recommandés pour vous » sur la page d'accueil, basée sur l'historique d'achats et de navigation de l'utilisateur.
- Implémentation progressive : v1 basée sur les tags/catégories, v2 avec embeddings pgvector.

2.3 Publication et Gestion des Agents

2.3.1 Processus de Publication

Le développeur suit un workflow guidé en 5 étapes pour publier un agent :

1. **Upload du fichier .agentjson** : Validation automatique du schéma côté client (feedback instantané) puis côté serveur (sécurité).
2. **Métadonnées** : Nom, description longue (Markdown), catégorie, tags (max 10), screenshots (max 5), vidéo de démo (optionnel).
3. **Configuration du mode** : Local (l'agent tourne sur la machine de l'utilisateur), Cloud (l'agent est hébergé par le développeur, URL endpoint obligatoire), ou Hybride (les deux modes disponibles).
4. **Tarification** : Gratuit, Achat unique (prix libre, min 0.99€), Abonnement mensuel (min 1.99€/mois), ou Pay-per-use (crédits, min 0.01€/interaction).
5. **Soumission** : L'agent passe en statut « pending ». Le pipeline de validation automatisée se déclenche. Si validation OK, l'agent passe en « approved » (ou « manual_review » si cas limite).

2.3.2 Pipeline de Validation Automatisée

Chaque agent soumis passe par un pipeline de vérification en 4 étapes, remplaçant la review manuelle systématique du CDC original.

1. **Validation de schéma** : Vérification stricte de la conformité au format .agentjson v2 (JSON Schema). Champs obligatoires, types, valeurs autorisées.
2. **Analyse de sécurité statique** : Détection de patterns dangereux dans le system_prompt (injections, tentatives d'exfiltration de clés, prompts manipulateurs). Scan des URLs d'endpoints (blacklist, réputation). Vérification que le fichier ne contient aucune clé API en dur.
3. **Vérification des permissions** : Les permissions déclarées sont cohérentes avec les tools et le system_prompt. Alerte si permissions excessives (ex : accès filesystem pour un agent de traduction).

4. **Test de fonctionnement** : Pour les agents cloud, un health check est effectué sur l'endpoint. Pour les agents locaux, un dry-run avec un prompt générique vérifie que la configuration est valide.

Règle de modération

Si les 4 étapes passent sans alerte : publication automatique (approved). Si une alerte est levée : l'agent est placé en file de review manuelle. Les admins reçoivent une notification et doivent statuer sous 48h. Le développeur est notifié par email à chaque changement de statut.

2.3.3 Gestion des Versions

- Chaque mise à jour crée une nouvelle entrée dans agent_versions avec un numéro de version sémantique (major.minor.patch).
- Changelog obligatoire pour chaque nouvelle version (champ texte Markdown).
- Les utilisateurs reçoivent une notification (push mobile + email optionnel) lors d'une mise à jour d'un agent possédé.
- Rollback possible : le développeur peut désactiver une version bugguée et revenir à la précédente.
- Les versions sont soumises au même pipeline de validation que la publication initiale.

2.3.4 Dashboard Développeur

- Vue d'ensemble : revenus totaux, revenus du mois, nombre d'agents actifs, nombre total de clients.
- Graphiques : évolution des ventes (line chart), répartition par agent (pie chart), tendance des notes (line chart).
- Par agent : vues de la fiche, taux de conversion (vue → achat), revenus générés, avis récents, taux de sandbox-to-purchase.
- Exports : téléchargement CSV des données de ventes et d'usage.
- Notifications : nouvel achat, nouvel avis, agent approuvé/rejeté, paiement envoyé.

2.4 Système de Paiements

2.4.1 Architecture Stripe Connect

AgentPlace utilise Stripe Connect en mode « Platform ». Chaque développeur doit créer un compte Stripe Connect (onboarding guidé intégré dans le dashboard) pour recevoir des paiements. La plateforme agit comme intermédiaire et prélève sa commission automatiquement.

2.4.2 Modèles de Tarification

Modèle	Fonctionnement	Commission AgentPlace	Reversement
Gratuit	Téléchargement libre, pas de paiement	0%	N/A

Modèle	Fonctionnement	Commission AgentPlace	Reversement
Achat unique	Paiement one-shot via Stripe Checkout	20%	Hebdomadaire (vendredi)
Abonnement mensuel	Stripe Subscriptions, renouvellement automatique	15%	Hebdomadaire (vendredi)
Pay-per-use	L'utilisateur achète des crédits, chaque interaction consomme N crédits	10%	Hebdomadaire (vendredi)

2.4.3 Gestion des Crédits (Pay-per-use)

- L'utilisateur achète des packs de crédits (5€ = 500 crédits, 20€ = 2200 crédits, 50€ = 6000 crédits) via Stripe Checkout.
- Chaque agent pay-per-use définit un coût par interaction (défini par le développeur, minimum 1 crédit).
- Le solde de crédits est affiché dans le header de l'app. Alerte quand le solde passe sous 50 crédits.
- Les crédits n'expirent jamais. Remboursement possible sur crédits non utilisés (politique de remboursement sous 14 jours).

2.4.4 Flux de Paiement Détaillé

1. L'utilisateur clique « Acheter » sur la fiche agent.
2. Redirection vers Stripe Checkout (ou modal intégré via Stripe Elements pour le mobile).
3. Paiement traité par Stripe. Webhook payment_intent.succeeded reçu côté API.
4. L'API crée l'entrée dans la table purchases, met à jour la bibliothèque de l'utilisateur.
5. Email de confirmation envoyé à l'utilisateur. Notification au développeur.
6. Le vendredi suivant, Stripe Transfer envoie automatiquement la part développeur (montant – commission).

2.4.5 Mode Sandbox (Essai Avant Achat)

Chaque agent payant offre un mode sandbox permettant à l'utilisateur de tester l'agent gratuitement avant achat. Le développeur configure les limites du sandbox dans le .agentjson (champ sandbox_config).

- Limite par défaut : 3 interactions maximum par utilisateur par agent.
- Le développeur peut personnaliser : nombre d'interactions, fonctionnalités activées, durée d'essai.
- Après épuisement du sandbox, l'utilisateur voit un CTA « Acheter pour continuer ».
- Le taux de conversion sandbox → achat est suivi dans le dashboard développeur.

2.5 Format Standard .agentjson v2

Le format .agentjson est le cœur technique d'AgentPlace. C'est un fichier JSON qui décrit complètement un agent IA : son identité, sa configuration, ses permissions, et ses métadonnées de distribution. Le format est versionné et rétrocompatible.

2.5.1 Schéma Complet

Champ	Type	Requis	Description
schema_version	string	Oui	Version du format (ex: « 2.0 »)
name	string	Oui	Nom de l'agent (3-60 caractères, unique par développeur)
version	string	Oui	Version sémantique de l'agent (ex: « 1.2.0 »)
mode	enum	Oui	local cloud hybrid
model	string string[]	Oui	ID du/des modèle(s) supportés (ex: « claude-sonnet-4-20250514 »)
endpoint	string null	Cloud	URL API sécurisée si mode cloud ou hybrid. HTTPS obligatoire.
system_prompt	string	Oui	Instructions système de l'agent (max 10 000 caractères)
tools	array	Non	Liste des outils MCP ou function calls (format OpenAI/Anthropic)
permissions	object	Oui	Déclaration des accès requis (voir 2.5.2)
config	object	Non	Paramètres du modèle (temperature, max_tokens, top_p, etc.)
metadata	object	Oui	Icône (URL), catégorie, tags[], description courte (max 200 car.)
dependencies	array	Non	Liste d'agents ou services requis pour fonctionner (IDs)
sandbox_config	object	Non	Limites sandbox : max_turns (défaut 3), enabled_features[], ttl_hours
health_check	string null	Cloud	URL de vérification de disponibilité (GET, doit retourner 200)
localization	object	Non	Traductions : { fr: { name, description }, en: { name, description } }

2.5.2 Système de Permissions

Chaque agent déclare explicitement les ressources auxquelles il a besoin d'accéder. L'utilisateur voit et approuve ces permissions avant la première utilisation (similaire au modèle Android/iOS).

Permission	Description	Niveau de risque
network	L'agent peut effectuer des requêtes HTTP sortantes	Moyen
filesystem.read	L'agent peut lire des fichiers locaux (drag & drop uniquement)	Moyen
filesystem.write	L'agent peut écrire des fichiers locaux	Élevé
clipboard	L'agent peut lire/écrire dans le presse-papier	Faible
user_data	L'agent peut accéder aux données du profil utilisateur	Moyen

Permission	Description	Niveau de risque
camera	L'agent peut accéder à la caméra (mobile)	Élevé
microphone	L'agent peut accéder au micro (conversation vocale)	Élevé
none	Aucun accès spécial requis (chat pur)	Aucun

2.6 Application Desktop (Tauri 2.0)

L'application desktop est le client principal pour l'exécution d'agents, en particulier les agents en mode local. Elle est construite avec Tauri 2.0 (Rust + React) pour garantir légèreté, performance native et sécurité.

2.6.1 Fonctionnalités Core

- **Authentification SSO** : Connexion liée au compte marketplace via OAuth flow (deep link). Le token est stocké dans le keychain OS natif (macOS Keychain, Windows Credential Manager, Linux Secret Service).
- **Bibliothèque d'agents** : Synchronisation avec l'API. Affiche tous les agents achetés/gratuits avec statut (installé, mise à jour disponible, cloud). Recherche et filtrage local.
- **Interface de chat** : UI de conversation unifiée, chaque agent a son propre thread. Support Markdown dans les réponses, coloration syntaxique pour le code, rendu LaTeX pour les formules.
- **Moteur d'exécution local** : Lit le .agentjson, construit la requête API vers le modèle IA (Claude, GPT-4, etc.) avec la clé API de l'utilisateur. Gère le streaming des réponses (SSE). Exécute les tools/function calls définis dans le .agentjson.
- **Connexion Cloud** : Pour les agents cloud, les requêtes sont relayées vers l'endpoint du développeur via un tunnel sécurisé (HTTPS + auth token).
- **Gestion des clés API** : Interface pour ajouter/modifier/supprimer ses clés API. Stockage chiffré dans le keychain OS natif. Les clés ne transitent jamais par les serveurs AgentPlace.

2.6.2 Fonctionnalités Avancées

- **Orchestration multi-agents** : Création de pipelines visuels (node editor) pour chaîner des agents. Ex : Agent Rédaction → Agent Relecture → Agent Traduction. L'output de chaque agent alimente l'input du suivant.
- **Agent Sélecteur** : Meta-agent intégré qui analyse la requête et route automatiquement vers le meilleur agent de la bibliothèque. Invocable via raccourci clavier global (configurable, défaut : Ctrl+Shift+A).
- **Raccourcis clavier globaux** : Invocation rapide de l'app ou d'un agent spécifique depuis n'importe quelle application. System tray avec accès rapide.
- **Drag & drop de fichiers** : Déposer un fichier dans le chat pour l'envoyer à l'agent (si permission filesystem.read accordée).

- **Historique persistant** : Toutes les conversations sont stockées localement (SQLite via Tauri) et recherchables en full-text.
- **Mise à jour automatique** : Check de mises à jour au démarrage et toutes les 6h. Download en arrière-plan. Notification à l'utilisateur.
- **Mode hors-ligne** : Les agents locaux utilisant des modèles exécutés via Ollama peuvent fonctionner sans connexion internet.

2.6.3 Plateformes Supportées

OS	Version minimum	Architecture	Distribution
macOS	12.0 (Monterey)	x64 + Apple Silicon (Universal)	.dmg signé + notarié (Apple Developer ID)
Windows	10 (21H2)	x64	.msi signé + NSIS installer
Linux	Ubuntu 22.04 / Fedora 38	x64	.deb + .ApplImage + Flathub (futur)

2.7 Application Mobile (React Native / Expo)

L'application mobile offre un accès aux agents Cloud depuis iOS et Android. Elle ne supporte pas l'exécution locale d'agents (limitation de puissance de calcul et de batterie sur mobile).

2.7.1 Fonctionnalités Core

- **Auth mobile** : deep links OAuth (GitHub, Google, Apple Sign-In natif), biométrie (Face ID / Touch ID / Fingerprint) pour le déverrouillage rapide.
- **Catalogue et recherche** : adaptation responsive du catalogue web, filtres accessibles via bottom sheet.
- **Achat in-app** : Stripe Elements intégré (pas d'achat via App Store/Google Play pour éviter la commission 30%).
- **Bibliothèque personnelle** : liste des agents achetés avec statut online/offline de l'endpoint cloud.
- **Chat mobile** : interface de conversation optimisée pour mobile, clavier avec boutons contextuels (joindre fichier, micro).
- **Notifications push** : nouveaux agents recommandés, mises à jour d'agents possédés, confirmations d'achat (Expo Notifications + APNs/FCM).

2.7.2 Fonctionnalités Spécifiques Mobile

- **Widgets natifs** : Widget iOS (WidgetKit) et Android (Glance) pour accéder à l'agent favori depuis l'écran d'accueil. Le widget affiche un champ de saisie rapide.
- **Raccourcis vocaux** : Intégration Siri Shortcuts (iOS) et Google Assistant Routines (Android) pour invoquer un agent par commande vocale.
- **Mode vocal** : Conversation mains libres : speech-to-text (Whisper / API native) pour l'input, text-to-speech (API native OS) pour la réponse de l'agent.

- **Partage natif** : Share Sheet (iOS) et Intent Share (Android) pour envoyer du texte/des fichiers directement à un agent.
- **Mode sombre** : Thème sombre natif suivant les préférences système.
- **Cache hors-ligne** : Les conversations précédentes sont accessibles hors connexion (cache SQLite local via expo-sqlite).

2.7.3 Plateformes Supportées

OS	Version minimum	Distribution
iOS	16.0	App Store (TestFlight pour bêta)
Android	API 26 (Android 8.0)	Google Play Store (Firebase App Distribution pour bêta)

2.8 Orchestration Multi-Agents (Agent Pipelines)

Fonctionnalité différenciante

L'orchestration multi-agents est LA fonctionnalité qui distingue AgentPlace de tous les concurrents. Aucune marketplace existante ne permet aux utilisateurs non-techniques de chaîner des agents dans des workflows automatisés.

2.8.1 Concept

Un pipeline est une séquence de 2 à 10 agents connectés entre eux. L'output de chaque agent alimente automatiquement l'input du suivant. L'utilisateur définit le pipeline une fois, puis peut l'exécuter en boucle avec différents inputs.

2.8.2 Interface de Création

- Éditeur visuel de type node graph (drag & drop) sur desktop. Version simplifiée (liste ordonnée) sur mobile.
- Chaque noeud représente un agent de la bibliothèque de l'utilisateur.
- Les connexions définissent le flux de données. Support des branchements conditionnels (si l'output contient X, aller vers agent A, sinon agent B).
- L'utilisateur peut configurer des transformations entre les noeuds (extraire un champ, reformater, etc.).
- Templates pré-configurés disponibles (Content Pipeline, Data Pipeline, Support Pipeline).

2.8.3 Moteur d'Exécution

- Exécution séquentielle par défaut : Agent 1 → output → Agent 2 → output → Agent 3.
- Exécution parallèle optionnelle : plusieurs agents reçoivent le même input et leurs outputs sont agrégés.
- Gestion des erreurs : si un agent échoue, le pipeline s'arrête et l'utilisateur est notifié avec le détail de l'erreur.
- Logs d'exécution : chaque étape est loggée avec input/output/durée pour le debugging.

- Les pipelines peuvent être sauvegardés, partagés (public/privé) et même vendus sur la marketplace.

2.9 Agent Sélecteur Intelligent

L'Agent Sélecteur est un meta-agent intégré à l'app desktop et mobile. Il analyse la requête de l'utilisateur et recommande automatiquement le meilleur agent de sa bibliothèque personnelle.

2.9.1 Fonctionnement

1. L'utilisateur invoque le Sélecteur (raccourci clavier global sur desktop, widget ou tap sur mobile).
2. Il tape sa requête en langage naturel (ex : « Traduis ce texte en anglais »).
3. Le Sélecteur analyse la requête et la compare aux descriptions/tags de tous les agents installés (cosine similarity sur embeddings).
4. Il propose le ou les agents les plus pertinents avec un score de confiance.
5. L'utilisateur sélectionne un agent et la conversation démarre directement.

2.9.2 Implémentation Technique

- Les descriptions et tags de chaque agent installé sont transformés en embeddings au moment de l'installation.
- La requête utilisateur est également transformée en embedding (via l'API du modèle configuré).
- La correspondance se fait par cosine similarity locale (pas d'appel serveur, tout en local pour la latence).
- Fallback : si aucun agent ne dépasse un seuil de confiance de 0.6, proposer une recherche sur la marketplace.

3. Architecture Technique

3.1 Vue d'Ensemble

L'architecture suit un modèle client-serveur classique avec une API centralisée, une base de données PostgreSQL, et des clients multiplateformes. Le choix technologique privilégie la productivité d'une équipe de 2 développeurs tout en garantissant la scalabilité future.

3.2 Stack Technologique Détaillé

Couche	Technologie	Version	Justification
Frontend Web	Next.js (App Router)	14.x	SSR pour SEO, RSC pour performance, écosystème React
UI Framework	Tailwind CSS + shadcn/ui	3.x / latest	Design system cohérent, composants accessibles, personnalisable
Backend API	Node.js + Fastify	22 LTS / 4.x	Plus performant qu'Express, validation de schéma native (Ajv), plugins
ORM	Drizzle ORM	latest	Type-safe, léger, migrations SQL natives, pas d'abstraction lourde
Base de données	PostgreSQL via Supabase	15+	RLS natif, Auth intégré, realtime, pgvector pour embeddings
Cache	Redis via Upstash	latest	Cache catalogue, sessions, rate limiting distribué, serverless-friendly
Queue de jobs	BullMQ (Redis)	5.x	Jobs async : paiements, emails, analytics, validation agents
CDN	Cloudflare	N/A	Assets statiques, protection DDoS, edge caching, WAF
Paiements	Stripe Connect	API v2024+	Standard marketplace, Connect pour les reversements, Checkout, Subscriptions
App Desktop	Tauri	2.x	Léger (~10MB vs 200MB Electron), Rust backend, sécurité native
App Mobile	React Native + Expo	SDK 52+	Code partagé iOS/Android, OTA updates, Expo Notifications
Stockage fichiers	Supabase Storage + Cloudflare R2	N/A	.agentjson et assets, CDN pour médias, coût réduit
Auth	Supabase Auth	v2	OAuth (GitHub/Google/Apple), JWT, RLS policies
Emails	Resend	latest	Emails transactionnels, templates React Email, délivrabilité
Monitoring	Sentry	latest	Error tracking frontend + backend, performance monitoring
Logs & Uptime	Better Stack	N/A	Logs structurés, uptime monitoring, alertes
Analytics produit	PostHog (cloud)	latest	Product analytics, feature flags, session replay, funnels

Couche	Technologie	Version	Justification
Déploiement web	Vercel	N/A	Preview deploys, edge functions, analytics
Déploiement API	Fly.io	N/A	Containers Docker, déploiement global multi-région, auto-scaling

3.3 Modèle de Données

Le schéma de base de données est géré par Drizzle ORM avec des migrations SQL versionnées. Les Row Level Security (RLS) politiques de Supabase sont utilisées pour sécuriser l'accès aux données.

Table	Champs principaux	Relations / Index
users	id (uuid PK), email (unique), display_name, avatar_url, role (enum: user dev admin), stripe_account_id, team_id (FK nullable), api_keys_encrypted (jsonb), preferences (jsonb), created_at, updated_at	Index : email. RLS : chaque user voit uniquement son propre profil.
teams	id (uuid PK), name, owner_id (FK users), plan (enum: free team enterprise), stripe_customer_id, sso_config (jsonb), created_at	Index : owner_id. RLS : membres de l'équipe uniquement.
agents	id (uuid PK), dev_id (FK users), name, slug (unique), mode (enum: local cloud hybrid), models (text[]), config_url, price (decimal), pricing_model (enum: free one_time subscription pay_per_use), credit_cost (int), status (enum: draft pending approved rejected suspended), rating (decimal), review_count (int), download_count (int), permissions (jsonb), category, tags (text[]), embedding (vector(1536)), created_at, updated_at	Index : slug, dev_id, category, status. GIN index sur tags. pgvector index (ivfflat) sur embedding.
agent_versions	id (uuid PK), agent_id (FK agents), version (semver), config_url, changelog (text), security_scan_status (enum), is_active (bool), created_at	Index : agent_id, version. Unique constraint (agent_id, version).
purchases	id (uuid PK), user_id (FK users), agent_id (FK agents), amount (decimal), currency (char(3)), stripe_payment_id, created_at	Index : user_id, agent_id. Unique constraint (user_id, agent_id) pour les achats uniques.
subscriptions	id (uuid PK), user_id (FK users), agent_id (FK agents), stripe_sub_id, status (enum: active cancelled past_due), current_period_end, created_at	Index : user_id, stripe_sub_id.
reviews	id (uuid PK), user_id (FK users), agent_id (FK agents), rating (int 1-5), comment (text), verified_purchase (bool), helpful_count (int), created_at, updated_at	Index : agent_id, user_id. Unique constraint (user_id, agent_id).
collections	id (uuid PK), user_id (FK users), name, description, is_public (bool), agent_ids (uuid[]), created_at	Index : user_id, is_public.
pipelines	id (uuid PK), user_id (FK users), name, description, agent_sequence (jsonb), config (jsonb), is_public (bool), price (decimal nullable), created_at	Index : user_id, is_public.

Table	Champs principaux	Relations / Index
usage_credits	id (uuid PK), user_id (FK users), balance (int), last_topup (timestamp), created_at	Index : user_id. Unique constraint (user_id).
usage_logs	id (uuid PK), user_id (FK users), agent_id (FK agents), credits_consumed (int), pipeline_id (FK nullable), created_at	Index : user_id, agent_id, created_at. Partitionnée par mois.
audit_logs	id (uuid PK), team_id (FK teams nullable), user_id (FK users), action (string), resource_type (string), resource_id (uuid), metadata (jsonb), created_at	Index : team_id, user_id, created_at. Partitionnée par mois. Rétention 12 mois.

3.4 Architecture API

L'API REST est construite avec Fastify et suit les conventions RESTful. Toutes les routes sont préfixées par /api/v1. L'authentification est gérée par JWT (Supabase Auth) passé dans le header Authorization: Bearer <token>.

Groupe	Endpoints principaux	Auth requise
Auth	POST /auth/signup, POST /auth/login, POST /auth/refresh, POST /auth/logout, POST /auth/forgot-password	Non (sauf logout)
Users	GET /users/me, PATCH /users/me, GET /users/:id (profil public), PUT /users/me/api-keys	Oui
Agents	GET /agents (catalogue), GET /agents/:slug, POST /agents (créer), PATCH /agents/:id, DELETE /agents/:id, POST /agents/:id/versions	Partiel (GET public, POST/PATCH/DELETE dev)
Search	GET /search?q=&category=&model=&price_min=&price_max=&sort=	Non
Purchases	POST /purchases (créer checkout session), GET /purchases/me (bibliothèque)	Oui
Subscriptions	POST /subscriptions, DELETE /subscriptions/:id, GET /subscriptions/me	Oui
Credits	POST /credits/topup, GET /credits/balance, GET /credits/history	Oui
Reviews	POST /agents/:id/reviews, GET /agents/:id/reviews, PATCH /reviews/:id, DELETE /reviews/:id	Partiel
Collections	GET /collections/me, POST /collections, PATCH /collections/:id, DELETE /collections/:id	Oui
Pipelines	GET /pipelines/me, POST /pipelines, PATCH /pipelines/:id, DELETE /pipelines/:id, POST /pipelines/:id/execute	Oui
Analytics	GET /analytics/dashboard (dev), GET /analytics/agents/:id (dev), GET /analytics/admin (admin)	Oui (dev/admin)
Admin	GET /admin/agents/pending, PATCH /admin/agents/:id/status, GET /admin/users, GET /admin/metrics	Oui (admin)
Webhooks	POST /webhooks/stripe (Stripe events)	Signature Stripe

4. Contraintes & Exigences Non-Fonctionnelles

4.1 Sécurité

Exigence	Implémentation	Responsable
Chiffrement des clés API	AES-256-GCM côté serveur, clé maître dans variable d'environnement, rotation semestrielle	Dev B
Chiffrement local (Desktop)	Stockage dans le keychain OS natif (macOS Keychain, Windows Credential Manager)	Dev A
Authentification	JWT via Supabase Auth, access token 1h, refresh token 7j, rotation automatique	Dev B
Rate limiting	Redis-based : Free 60 req/min, Dev 120 req/min, Admin illimité. Par IP + par user	Dev B
Protection web	Cloudflare WAF, CSP headers stricts, HSTS, X-Frame-Options, protection XSS/CSRF (tokens CSRF)	Dev B
Validation des agents	Pipeline automatisé en 4 étapes (schema, sécurité, permissions, health check)	Dev B
HTTPS	Certificats TLS sur tous les endpoints (Cloudflare + Let's Encrypt). Pas de HTTP.	Dev B
Secrets management	Variables d'environnement via Vercel/Fly.io. Aucun secret en dur dans le code. Rotation trimestrielle.	Dev B
Dépendances	Dependabot actif, audit npm hebdomadaire, verrouillage des versions (lockfile)	Les deux

4.2 Performance

Métrique	Cible	Mesure
Time to First Byte (TTFB)	< 200ms (catalogue, pages statiques)	Vercel Analytics + Lighthouse CI
Largest Contentful Paint (LCP)	< 2.5s	Lighthouse CI dans la pipeline GitHub Actions
First Input Delay (FID)	< 100ms	Lighthouse CI
Cumulative Layout Shift (CLS)	< 0.1	Lighthouse CI
Lighthouse Score global	> 90	Lighthouse CI automatique sur chaque PR
Bundle JS (First Load)	< 200KB (gzipped)	Analyse bundle avec next/bundle-analyzer
Temps de chargement agent local	< 500ms	Benchmark Tauri intégré
Temps de réponse API (p95)	< 300ms	Better Stack monitoring

Métrique	Cible	Mesure
Images	WebP/AVIF avec lazy loading natif	Next/Image + Cloudflare Image Resizing

4.3 Observabilité et Monitoring

- **Error tracking** : Sentry sur le frontend web, le backend API, l'app desktop et l'app mobile. Alertes Slack pour les erreurs critiques (> 5 occurrences/h).
- **Logs structurés** : Better Stack pour les logs JSON du backend. Rétention 30 jours. Recherche full-text.
- **Uptime monitoring** : Better Stack avec checks toutes les 60s sur les endpoints critiques (API health, web, Stripe webhook). Alertes SMS + Slack en cas de downtime.
- **Analytics produit** : PostHog pour les events clés (signup, first_purchase, sandbox_start, pipeline_created, etc.), funnels de conversion, feature flags.
- **Status page** : Page publique (Better Stack) affichant l'état de chaque service (API, Web, Desktop updates, Mobile, Payments).

4.4 Accessibilité

- Conformité WCAG 2.1 niveau AA minimum.
- Navigation au clavier complète sur le web.
- Labels ARIA sur tous les éléments interactifs.
- Contraste de couleurs minimum 4.5:1 (texte) et 3:1 (grands textes).
- Support des lecteurs d'écran (VoiceOver, NVDA, TalkBack).
- Tests automatisés avec axe-core dans la CI.

4.5 Légal et Conformité

- **RGPD** : Bannière de consentement cookies (Cookiebot ou solution custom). Droit d'accès, de rectification, de suppression et de portabilité des données. Données hébergées en UE (Supabase région eu-west, Fly.io Paris).
- **Mentions légales** : Conformes au droit français (loi LCEN). Identité de l'éditeur, hébergeur, directeur de publication.
- **CGU** : Conditions générales d'utilisation couvrant : inscription, utilisation des agents, paiements, propriété intellectuelle, responsabilité des développeurs sur le contenu de leurs agents.
- **CGV** : Conditions générales de vente spécifiques à la marketplace : commission, reversements, remboursements, litiges.
- **Politique de confidentialité** : Détail des données collectées, finalités, durées de conservation, droits des utilisateurs, DPO contact.
- **PI des agents** : Les développeurs conservent la propriété intellectuelle de leurs agents. AgentPlace dispose d'une licence d'utilisation pour la distribution. Contenu généré par les agents : responsabilité de l'utilisateur final.

- **Modèles IA tiers** : L'utilisateur utilise sa propre clé API. AgentPlace n'est pas responsable des coûts ou des limitations imposées par les fournisseurs de modèles.

4.6 Disaster Recovery

Aspect	Stratégie	Fréquence / Cible
Backup base de données	Supabase Point-in-Time Recovery + export pg_dump quotidien vers R2	Quotidien, rétention 30 jours
Backup fichiers	Réplication Supabase Storage vers Cloudflare R2	Temps réel
RTO (Recovery Time Objective)	Restauration complète de la plateforme	< 1 heure
RPO (Recovery Point Objective)	Perte de données maximale acceptable	< 15 minutes
Runbook incidents	Documentation des procédures de récupération pour chaque scénario	Mis à jour trimestriellement
Tests de restauration	Restauration complète en environnement de staging	Mensuel

5. Découpage en Sprints

Organisation : sprints de 2 semaines, équipe de 2 développeurs. Les points d'effort sont en Fibonacci (1, 2, 3, 5, 8, 13) et représentent la complexité relative. Capacité estimée par sprint : 30-35 points pour 2 développeurs.

Répartition des tâches

Chaque user story est assignée à un développeur (A = Frontend/Apps, B = Backend/Infra).
Les tâches critiques (paiements, sécurité, moteur d'exécution) sont travaillées en pair programming.
Code review croisée obligatoire sur chaque PR avant merge.

Sprint 0 — Fondations & Setup | Semaines 1-2

Objectif : Environnement de développement opérationnel, architecture posée, repository configuré, outils de monitoring en place.

#	User Story	Prio	Pts	Assigné
S0-1	Initialisation du monorepo (pnpm workspaces) : packages web (Next.js), api (Fastify), shared (types, utils), desktop (Tauri), mobile (Expo)	Must	5	A + B
S0-2	Configuration Supabase : création du projet, schéma DB initial (Drizzle migrations), Auth providers (GitHub, Google, Apple), Storage buckets, RLS policies de base	Must	8	B
S0-3	CI/CD : GitHub Actions (lint ESLint + Prettier, type-check, tests Vitest, build, preview deploys Vercel), branch protection rules	Must	5	B

#	User Story	Prio	Pts	Assigné
S0-4	Définition et documentation du format .agentjson v2 : JSON Schema, exemples, validation library (Zod), documentation sur GitHub Wiki	Must	3	A
S0-5	Setup monitoring : Sentry (web + API), Better Stack (logs + uptime), PostHog (analytics), Cloudflare (DNS + CDN + WAF)	Must	5	B
S0-6	Maquettes Figma : wireframes des pages clés (accueil, catalogue, fiche agent, dashboard dev, chat), design system (couleurs, typo, composants de base)	Should	5	A
S0-7	Setup Redis (Upstash) : connexion, helpers de cache, configuration rate limiter (express-rate-limit + Redis store)	Must	3	B

Sprint 1 — Authentification & Profils | Semaines 3-4

Objectif : Les utilisateurs peuvent s'inscrire, se connecter, gérer leur profil. Le système de rôles est opérationnel.

#	User Story	Prio	Pts	Assigné
S1-1	Backend Auth : endpoints signup/login/logout/refresh/forgot-password, middleware JWT Fastify, validation Zod	Must	5	B
S1-2	Frontend Auth : pages inscription/connexion/récupération MDP, formulaires avec react-hook-form + Zod, gestion du state auth (Zustand ou Context)	Must	5	A
S1-3	OAuth GitHub, Google et Apple via Supabase Auth : configuration providers, callbacks, gestion des profils OAuth	Must	3	B
S1-4	Page profil utilisateur : avatar (upload Supabase Storage), bio, préférences, gestion des clés API (UI masquée + chiffrement AES-256 backend)	Must	5	A + B
S1-5	Page profil développeur (publique) : bio, liens, portfolio d'agents, note moyenne, badge, compteurs	Must	5	A
S1-6	Sélection type de compte (user/dev) à l'inscription + upgrade user → dev depuis les paramètres	Must	2	A
S1-7	Gestion des sessions : refresh token, déconnexion de toutes les sessions, middleware de protection des routes	Must	3	B

Sprint 2 — Catalogue & Découverte | Semaines 5-6

Objectif : Les utilisateurs peuvent parcourir, rechercher, filtrer et consulter les agents de la marketplace.

#	User Story	Prio	Pts	Assigné
S2-1	Page d'accueil : hero section avec recherche, sections vedette/tendances/nouveautés, grille de catégories. Données servies par API avec cache Redis (TTL 5min)	Must	8	A + B
S2-2	Catalogue avec filtres : page catalogue, cards agents, filtres combinables (catégorie, modèle, prix, note, mode), tri, URL query params synchronisés	Must	8	A

#	User Story	Prio	Pts	Assigné
S2-3	Moteur de recherche full-text : endpoint GET /search, ts_vector + ts_query PostgreSQL, ranking par pertinence + popularité, cache Redis	Must	5	B
S2-4	Fiche détaillée d'un agent : description Markdown, screenshots, metadata, permissions, avis (placeholder), boutons d'action, SEO (metadata dynamiques)	Must	5	A
S2-5	Pagination et lazy loading : infinite scroll avec Intersection Observer, skeleton loaders, 20 agents par page	Must	3	A
S2-6	Collections et favoris : CRUD collections, bouton favori, page « Mes Favoris », API endpoints	Should	5	A + B

Sprint 3 — Publication d'Agents | Semaines 7-8

Objectif : Les développeurs peuvent publier, mettre à jour et gérer leurs agents sur la marketplace.

#	User Story	Prio	Pts	Assigné
S3-1	Dashboard développeur : layout avec sidebar, navigation, page d'accueil dashboard avec compteurs et graphiques (Recharts)	Must	5	A
S3-2	Formulaire de création d'agent : wizard multi-étapes (upload .agentjson, métadonnées, mode, tarification), validation client Zod, upload screenshots	Must	8	A
S3-3	Pipeline de validation serveur : validation schéma JSON, analyse sécurité statique, vérification permissions, health check endpoints cloud, job BullMQ async	Must	8	B
S3-4	Gestion des versions : upload nouvelle version, changelog obligatoire, rollback, re-validation pipeline	Should	5	B
S3-5	Panel admin modération : liste des agents pending, interface approve/reject avec motif, notifications email au développeur (Resend)	Must	5	A + B
S3-6	Calcul embeddings : job BullMQ qui génère les embeddings de chaque agent approuvé et les stocke dans pgvector pour la recherche sémantique et les recommandations	Should	3	B

Sprint 4 — Paiements & Acquisitions | Semaines 9-10

Objectif : Parcours d'achat complet fonctionnel. Les développeurs reçoivent leurs paiements.

#	User Story	Prio	Pts	Assigné
S4-1	Stripe Connect onboarding : flux d'inscription développeur, verification, dashboard Stripe lié dans le dashboard dev	Must	8	B
S4-2	Achat unique : Stripe Checkout Session, webhook payment_intent.succeeded, création purchase, mise à jour bibliothèque, email confirmation (Resend)	Must	8	B
S4-3	Abonnements : Stripe Subscriptions, webhooks (invoice.paid, customer.subscription.updated, customer.subscription.deleted), gestion statuts	Must	8	B

#	User Story	Prio	Pts	Assigné
S4-4	Système de crédits : achat de packs via Stripe Checkout, table usage_credits, débit à chaque interaction, affichage solde dans le header	Must	5	A + B
S4-5	Bibliothèque personnelle : page « Mes Agents » avec statut (acheté/abonné/gratuit), filtres, lien vers le chat	Must	3	A
S4-6	Mode Sandbox : 3 interactions gratuites par agent payant, compteur côté serveur, CTA achat après épuisement, tracking PostHog du funnel sandbox → purchase	Must	5	A + B

Sprint 5 — App Desktop — Fondations | Semaines 11-12

Objectif : Application Tauri fonctionnelle avec auth, bibliothèque et interface de chat de base.

#	User Story	Prio	Pts	Assigné
S5-1	Setup projet Tauri 2.0 : scaffold React + Tauri, configuration build multi-OS (macOS, Windows, Linux), signatures, auto-updater	Must	5	A
S5-2	Auth desktop : OAuth flow via deep links (tauri-plugin-deep-link), stockage token dans keychain OS natif (tauri-plugin-store chiffré)	Must	5	A
S5-3	Bibliothèque desktop : sync avec l'API, affichage agents achetés, statut (installé/mise à jour/cloud), UI de gestion	Must	5	A
S5-4	Interface de chat : UI conversation (messages, input, streaming), support Markdown + code highlight (react-markdown + Prism), gestion des threads par agent	Must	8	A
S5-5	Stockage local : base SQLite (tauri-plugin-sql) pour historique conversations, recherche full-text	Must	5	A
S5-6	System tray + raccourcis : icône system tray, raccourci global configurable (Ctrl+Shift+A), lancement au démarrage (optionnel)	Should	3	A

Sprint 6 — App Desktop — Moteur d'Exécution | Semaines 13-14

Objectif : Les agents locaux et cloud sont exécutables depuis l'app desktop. Gestion des clés API locale.

#	User Story	Prio	Pts	Assigné
S6-1	Moteur d'exécution local : parser le .agentjson, construire la requête API vers le modèle IA configuré (Claude, GPT-4, Gemini, Mistral), gérer le streaming SSE des réponses	Must	8	B
S6-2	Support multi-modèles : adaptateurs pour les APIs Anthropic, OpenAI, Google AI, Mistral. Interface commune pour le moteur d'exécution	Must	8	B
S6-3	Connexion agents Cloud : appel sécurisé vers l'endpoint du développeur, auth token, gestion timeout et retry	Must	5	B
S6-4	Gestion clés API locales : UI d'ajout/modification/suppression, stockage chiffré dans le keychain OS, les clés ne transitent jamais par les serveurs AgentPlace	Must	5	A + B

#	User Story	Prio	Pts	Assigné
S6-5	Drag & drop fichiers : déposer un fichier dans le chat, lecture et envoi au modèle (si permission filesystem.read), prévisualisation du fichier	Should	3	A
S6-6	Mise à jour agents : check périodique des nouvelles versions via API, notification utilisateur, téléchargement en arrière-plan	Should	3	A

Sprint 7 — Avis, Analytics & Recommandations | Semaines 15-16

Objectif : Système de feedback opérationnel. Dashboard analytics complet pour les développeurs. Recommandations personnalisées.

#	User Story	Prio	Pts	Assigné
S7-1	Système de notation : composant stars rating, vérification achat, formulaire commentaire, modération (signalement), calcul note moyenne (trigger PostgreSQL)	Must	5	A + B
S7-2	Affichage des avis : section avis sur fiche agent, distribution des notes (bar chart), tri (récents/utiles), pagination, bouton « Utile »	Must	5	A
S7-3	Dashboard analytics développeur : graphiques revenus (Recharts), ventes par agent, vues vs achats (funnel), évolution des notes, top agents	Must	8	A
S7-4	Recommandations personnalisées : section « Agents similaires » (cosine similarity pgvector), section « Recommandés pour vous » (historique user)	Should	5	B
S7-5	Emails transactionnels (Resend + React Email) : confirmation achat, nouvel avis, agent approuvé/rejeté, paiement envoyé, mise à jour agent	Should	3	B
S7-6	Exports CSV : téléchargement des données de ventes, historique transactions, liste des avis (dashboard dev)	Could	2	B

Sprint 8 — Application Mobile MVP | Semaines 17-18

Objectif : Application React Native fonctionnelle sur iOS et Android. Catalogue, achat et chat agents Cloud.

#	User Story	Prio	Pts	Assigné
S8-1	Setup Expo (SDK 52+) : navigation (expo-router, Stack + Tabs), thème (mode sombre natif), configuration assets et splash screen	Must	3	A
S8-2	Auth mobile : deep links OAuth, Apple Sign-In natif, biométrie (expo-local-authentication), gestion session (expo-secure-store)	Must	5	A
S8-3	Catalogue mobile : adaptation responsive, recherche, filtres (bottom sheet), cards agents, fiche détaillée, pagination infinite scroll	Must	8	A
S8-4	Achats mobile : intégration Stripe (via WebView Stripe Checkout ou @stripe/stripe-react-native), bibliothèque personnelle	Must	5	A

#	User Story	Prio	Pts	Assigné
S8-5	Chat mobile : interface conversation optimisée, clavier avec boutons contextuels, streaming des réponses, support Markdown	Must	8	A
S8-6	Notifications push : Expo Notifications + APNs/FCM, notifications nouvel agent, mise à jour, confirmation achat	Should	3	B

Sprint 9 — Orchestration Multi-Agents | Semaines 19-20

Objectif : Les utilisateurs peuvent créer et exécuter des pipelines multi-agents. Agent Sélecteur opérationnel.

#	User Story	Prio	Pts	Assigné
S9-1	Modèle de données pipelines + API CRUD : table pipelines, endpoints REST, validation des séquences d'agents (vérifier que l'utilisateur possède tous les agents référencés)	Must	5	B
S9-2	Interface de création pipeline (Desktop) : éditeur visuel node graph (React Flow), drag & drop agents, connexions, configuration des transformations inter-noeuds	Must	8	A
S9-3	Moteur d'exécution pipeline : exécution séquentielle et parallèle, passage d'output en input, gestion d'erreurs (stop + notification), logs d'exécution détaillés	Must	8	B
S9-4	Agent Sélecteur Intelligent : calcul embeddings des agents installés au moment de l'installation, matching cosme similarity local, UI de sélection rapide (desktop + mobile)	Should	5	A + B
S9-5	Templates de pipelines pré-configurés : Content Pipeline, Data Analysis Pipeline, Support Pipeline. Galerie de templates sur le web	Should	3	A

Sprint 10 — Fonctionnalités Mobile Avancées & Sprint Buffer | Semaines 21-22

Objectif : Fonctionnalités mobiles spécifiques. Absorption de la dette technique accumulée.

#	User Story	Prio	Pts	Assigné
S10-1	Widgets natifs iOS (WidgetKit via expo-widgets) et Android (Glance) : champ de saisie rapide pour l'agent favori	Should	5	A
S10-2	Mode vocal mobile : speech-to-text (expo-speech ou Whisper API) pour l'input, text-to-speech natif pour la réponse de l'agent	Should	5	A
S10-3	Siri Shortcuts (iOS) et Google Assistant Routines (Android) : invoquer un agent par commande vocale	Could	5	A
S10-4	Refactoring et dette technique : code review global, cleanup, résolution des TODO, amélioration de la couverture de tests	Must	8	A + B
S10-5	Documentation technique : README complet, architecture decision records (ADR), guide de contribution, documentation API (Swagger/OpenAPI auto-généré)	Must	5	A + B

Sprint 11 — Sécurité, Polish & Lancement | Semaines 23-24

Objectif : Plateforme prête pour le lancement public. Sécurité audité, tests complets, pages légales.

#	User Story	Prio	Pts	Assigné
S11-1	Audit de sécurité : checklist OWASP Top 10, tests d'injection SQL/XSS/CSRF, vérification auth bypass, scan de dépendances (npm audit + Snyk), pen testing manuel sur les endpoints critiques	Must	8	B
S11-2	Tests end-to-end : Playwright pour le web (parcours inscription → achat → chat), tests API intégration (Vitest + supertest), tests Detox pour mobile (parcours critique)	Must	8	A + B
S11-3	SEO et marketing : meta tags dynamiques (Next.js generateMetadata), sitemap.xml, robots.txt, OG tags, Twitter cards, structured data (JSON-LD pour les agents)	Must	3	A
S11-4	Pages légales : CGU, CGV, Politique de Confidentialité, Mentions Légales. Rédaction conforme au droit français et RGPD.	Must	3	A
S11-5	Onboarding développeur : guide pas-à-pas intégré (tooltips, wizard), documentation « Publier votre premier agent en 5 minutes », exemples .agentjson	Should	5	A
S11-6	Landing page et blog : page marketing avec value proposition, témoignages, pricing, CTA. Blog avec 3 articles SEO initiaux (Next.js MDX)	Could	5	A
S11-7	Status page publique (Better Stack) + runbook incidents (Notion/Wiki) + procédure de rollback documentée	Must	3	B

6. Roadmap et Jalons

Phase	Sprints	Durée	Livrable	Critère Go/No-Go
Phase 0 — Setup	Sprint 0	2 sem.	Env. technique complet, CI/CD, monitoring	Build + deploy automatisé, DB migrée, Sentry actif
Phase 1 — Core Web	Sprints 1 à 4	8 sem.	Marketplace web fonctionnelle avec paiements	Parcours complet inscription → achat → bibliothèque testé E2E
Phase 2 — Desktop	Sprints 5-6	4 sem.	App Tauri : auth, chat, exécution locale et cloud	Agent local exécuté avec clé API utilisateur, réponse streamée
Phase 3 — Feedback	Sprint 7	2 sem.	Avis, analytics dev, recommandations	Dashboard dev avec données réelles, recommandations fonctionnelles
Phase 4 — Mobile	Sprint 8	2 sem.	App React Native iOS + Android	Chat agent cloud fonctionnel sur iOS et Android
Phase 5 — Multi-Agents	Sprint 9	2 sem.	Orchestration multi-agents, Agent Sélecteur	Pipeline de 3 agents exécuté avec succès de bout en bout
Phase 6 — Polish	Sprint 10	2 sem.	Mobile avancé, dette technique, documentation	Couverture tests > 70%, documentation API complète

Phase	Sprints	Durée	Livrable	Critère Go/No-Go
Phase 7 — Launch	Sprint 11	2 sem.	Sécurité, SEO, légal, production	Audit sécurité passé, E2E pass, uptime > 99.5% sur 48h

Planning global

Durée totale : 12 sprints × 2 semaines = 24 semaines (6 mois).

Démarrage estimé : Avril 2026. Lancement production : Septembre 2026.

Ce planning est indicatif et suppose 2 développeurs à temps plein. Des ajustements seront faits à chaque rétrospective de sprint.

7. Critères de Succès

Indicateur	Cible à 3 mois post-lancement	Cible à 6 mois	Outil de mesure
Agents publiés	20+ agents approuvés	80+ agents	Query DB (status=approved)
Développeurs inscrits	30+ comptes développeurs	150+	Supabase Auth (role=dev)
Utilisateurs actifs mensuels (MAU)	200+	1 000+	PostHog (sessions uniques mensuelles)
Taux d'erreur paiements	< 0.5%	< 0.3%	Stripe Dashboard + alertes Better Stack
Score Lighthouse	> 90	> 95	Lighthouse CI automatisé sur chaque PR
Uptime	> 99.5%	> 99.9%	Better Stack monitoring (60s intervals)
Rétention J7	> 30%	> 45%	PostHog cohortes
NPS (Net Promoter Score)	> 40	> 50	Enquête in-app trimestrielle (PostHog surveys)
Temps moyen avant premier achat	< 15 min	< 10 min	Funnel PostHog (signup → purchase)
Conversion sandbox → achat	> 15%	> 25%	Events PostHog (sandbox_complete → purchase)
Téléchargements app desktop	100+	500+	GitHub Releases + PostHog
Téléchargements app mobile	200+	1 000+	App Store Connect + Google Play Console

8. Modèle Économique

8.1 Sources de Revenus

Source	Modèle	Détail
Commission achats uniques	20% par transaction	Prélevé automatiquement via Stripe Connect application_fee
Commission abonnements	15% récurrent	Prélevé sur chaque paiement mensuel via Stripe
Commission pay-per-use	10% sur les crédits consommés	Débit de la part plateforme lors du reversement hebdomadaire
Plans Team (futur, M+6)	29€/mois par siège	Bibliothèque partagée, rôles, facturation centralisée
Plans Enterprise (futur, M+12)	Sur devis (> 500€/mois)	SSO SAML, agents privés, audit log, SLA garanti
Agent Boosting (futur, M+6)	Placement sponsorisé	Les développeurs paient pour mettre en avant leurs agents

8.2 Coûts d'Opération Mensuels Estimés

Service	Plan	Coût/mois
Supabase (DB, Auth, Storage)	Pro	25 €
Vercel (Frontend)	Pro	20 €
Fly.io (API Backend)	2 shared-cpu-2x machines	30-50 €
Upstash (Redis)	Pay-as-you-go	5-15 €
Cloudflare (CDN, WAF, DNS)	Free / Pro	0-20 €
Sentry (Error tracking)	Team	26 €
Better Stack (Logs + Uptime)	Starter	24 €
PostHog (Analytics)	Free (1M events/mois)	0 €
Resend (Emails)	Free (3000/mois)	0 €
Domaine + DNS	agentplace.ai	15 €
Apple Developer Account	Annuel / 12	8 €
Google Play Developer	One-time / 12	2 €
TOTAL ESTIMÉ		155-205 €/mois

Seuil de rentabilité

Avec un coût fixe d'environ 180€/mois et une commission moyenne de 17%, AgentPlace atteint la rentabilité opérationnelle à partir d'environ 1 060€ de volume de transactions mensuel (soit environ 70 achats à 15€ en moyenne). Cet objectif est réaliste à M+3 post-lancement.

9. MVP — Stratégie de Lancement à Coût Zéro

Pourquoi un MVP avant le produit complet ?

Le CDC complet (sections 1 à 8) représente la vision cible d'AgentPlace. Mais lancer un projet de cette envergure sans validation marché est un risque.

Cette section définit un MVP (Minimum Viable Product) déployable en 4 semaines, avec un coût d'infrastructure de 0€/mois grâce aux tiers gratuits.

L'objectif : valider le concept, récolter les premiers utilisateurs et feedbacks, puis investir progressivement vers le produit complet.

Le MVP ne remplace pas le CDC complet — il le précède. Tout le code MVP sera réutilisé et enrichi dans les sprints suivants.

9.1 Périmètre du MVP

Le MVP se concentre sur le cœur de valeur d'AgentPlace : permettre à un développeur de publier un agent IA et à un utilisateur de le découvrir et l'utiliser. Tout le reste est volontairement exclu pour aller vite et à coût nul.

9.1.1 Ce qui est INCLUS dans le MVP

- **Marketplace web statique** : Landing page + catalogue d'agents + fiche détaillée. Pas de SSR lourd : Next.js en mode statique (SSG) déployé sur Vercel (gratuit).
- **Authentification** : Inscription / connexion email + GitHub OAuth via Supabase Auth (tier gratuit : 50 000 MAU).
- **Catalogue d'agents** : Recherche basique (full-text PostgreSQL natif), filtres par catégorie et modèle IA, pagination simple.
- **Publication d'agents** : Formulaire d'upload .agentjson simplifié, validation de schéma (Zod), review manuelle (acceptable à petite échelle).
- **Interface de chat web** : Chat intégré directement dans la fiche agent (pas d'app desktop/mobile). L'utilisateur fournit sa propre clé API. Le chat est exécuté côté client (la clé ne transite pas par le serveur).
- **Profils basiques** : Profil utilisateur (avatar, bio), profil développeur public (bio, liste d'agents).
- **Agents gratuits uniquement** : Pas de paiement dans le MVP. Tous les agents sont gratuits. Cela élimine Stripe et toute la complexité paiement.

9.1.2 Ce qui est EXCLU du MVP (reporté au CDC complet)

- Paiements Stripe / monétisation (reporté Sprint 4).
- Application desktop Tauri (reporté Sprints 5-6).
- Application mobile React Native (reporté Sprint 8).
- Orchestration multi-agents / pipelines (reporté Sprint 9).
- Agent Sélecteur Intelligent (reporté Sprint 9).
- Système de recommandations pgvector (reporté Sprint 7).

- Mode sandbox / essai avant achat (reporté Sprint 4).
- Système d'avis et notation (reporté Sprint 7).
- Collections et favoris (reporté Sprint 2).
- Emails transactionnels (reporté Sprint 7).
- Notifications push (reporté Sprint 8).
- Mode entreprise / SSO (futur).
- Redis cache / BullMQ queues (reporté Sprint 0 complet).
- CDN Cloudflare / WAF (reporté Sprint 0 complet).

Règle d'or du MVP

Si une fonctionnalité n'est pas indispensable pour que le parcours « Développeur publie un agent → Utilisateur le trouve et l'utilise en chat » fonctionne, elle est exclue du MVP. Aucune exception.

9.2 Stack Technique MVP (100% Gratuit)

Le MVP utilise exclusivement des services ayant un tier gratuit généreux, permettant un coût d'infrastructure de 0€/mois tant que le trafic reste modéré (< 1000 utilisateurs).

Couche	Service MVP	Tier gratuit	Limite	Remplacement CDC complet
Frontend	Next.js sur Vercel	Hobby (gratuit)	100 GB bandwidth/mois, 1 projet	Vercel Pro (20€/mois)
Backend API	Next.js API Routes (Vercel)	Inclus dans Hobby	Serverless functions, 100GB-hrs/mois	Fastify sur Fly.io
Base de données	Supabase	Free (gratuit)	500 MB, 2 projets, 50k MAU auth	Supabase Pro (25€/mois)
Auth	Supabase Auth	Inclus	50 000 MAU, OAuth providers illimités	Identique
Stockage fichiers	Supabase Storage	Inclus	1 GB stockage, 2 GB transfert/mois	Supabase Pro + R2
ORM	Drizzle ORM	Open source	Illimité	Identique
UI	Tailwind CSS + shadcn/ui	Open source	Illimité	Identique
Monitoring	Sentry	Developer (gratuit)	5k events/mois	Sentry Team (26€/mois)
Analytics	PostHog Cloud	Free	1M events/mois	Identique
Domaine	Vercel subdomain (.vercel.app)	Gratuit	Pas de custom domain	agentplace.ai (~15€/mois)
Emails	Resend	Free	3 000 emails/mois, 1 domaine	Identique

Coût total MVP : 0€/mois

Tous les services utilisés disposent d'un tier gratuit suffisant pour le MVP.

Le seul coût potentiel est le nom de domaine custom (~12€/an) mais il est optionnel : le MVP peut tourner sur agentplace.vercel.app.

Les limites les plus contraignantes sont : 500 MB de base de données (Supabase Free) et 100 GB de bandwidth (Vercel Hobby). Ces limites sont largement suffisantes pour valider le concept avec quelques centaines d'utilisateurs.

9.3 Architecture Simplifiée du MVP

L'architecture MVP élimine la complexité du CDC complet en fusionnant le frontend et le backend dans un seul déploiement Next.js sur Vercel. Pas de serveur séparé, pas de Redis, pas de queue de jobs.

9.3.1 Architecture Mono-Deployment

- **Next.js App Router** : Le frontend (pages, composants React) et le backend (API Routes / Server Actions) coexistent dans le même projet Next.js déployé sur Vercel.
- **Server Actions pour les mutations** : Création d'agent, mise à jour de profil, etc. sont gérées par des Server Actions Next.js (pas d'API REST séparée au stade MVP).
- **Supabase client direct** : Les requêtes DB passent par le client Supabase JS (côté serveur via Server Components / Server Actions). Les RLS politiques sécurisent l'accès.
- **Chat côté client** : L'interface de chat exécute les requêtes vers les APIs de modèles IA directement depuis le navigateur de l'utilisateur. Sa clé API ne transite jamais par le serveur AgentPlace. C'est un choix de sécurité et de simplicité.
- **Pas de cache Redis** : Le cache est géré nativement par Vercel (ISR + stale-while-revalidate). Suffisant pour le volume du MVP.
- **Pas de queue de jobs** : Les opérations async (validation d'agent, envoi d'email) sont exécutées de manière synchrone ou via des Vercel Cron Jobs (gratuit, 1/jour sur Hobby).

9.3.2 Modèle de Données MVP (réduit)

Le modèle de données MVP est un sous-ensemble strict du modèle complet. Les tables non utilisées ne sont pas créées. Elles seront ajoutées par migration lors des sprints suivants.

Table	Champs	Diff vs CDC complet
users	id, email, display_name, avatar_url, role (user dev admin), created_at	Pas de stripe_account_id, team_id, api_keys_encrypted, preferences
agents	id, dev_id, name, slug, mode (local cloud), model (text), config_url, status (draft pending approved rejected), category, tags, description, created_at, updated_at	Pas de price, pricing_model, rating, permissions, embedding, download_count
agent_versions	id, agent_id, version, config_url, changelog, created_at	Pas de security_scan_status, is_active

Tables absentes du MVP : purchases, subscriptions, reviews, collections, pipelines, usage_credits, usage_logs, audit_logs, teams. Elles seront ajoutées via des migrations Drizzle au fur et à mesure des sprints du CDC complet.

9.4 Fonctionnalités MVP Détaillées

9.4.1 Landing Page

- Hero section : tagline (« La marketplace ouverte d'agents IA »), barre de recherche, compteur d'agents, CTA « Parcourir les agents » / « Publier un agent ».
- Section « Comment ça marche » : 3 étapes visuelles (Découvrez → Installez → Utilisez).
- Grille de catégories avec icônes.
- Section « Derniers agents publiés » (6 cards dynamiques).
- Footer avec liens légaux (mentions légales, CGU basiques).
- Design : Tailwind CSS + shadcn/ui. Mode sombre natif (prefers-color-scheme).

9.4.2 Catalogue

- Page /agents : grille responsive de cards agents.
- Chaque card : nom, icône (ou placeholder), description courte (120 car. max), catégorie, modèle IA, badge « Gratuit ».
- Filtres : catégorie (select), modèle IA (select), recherche texte (input). Filtres synchronisés avec les query params URL.
- Recherche full-text via PostgreSQL natif (ts_vector sur name + description + tags). Pas de pgvector au stade MVP.
- Pagination simple (20 agents/page, boutons Précédent/Suivant). Pas d'infinite scroll au MVP.

9.4.3 Fiche Agent

- Page /agents/[slug] : description longue (Markdown rendu via react-markdown), modèle(s) supporté(s), mode (local/cloud), version actuelle, lien vers le profil développeur.
- Bouton « Utiliser cet agent » : ouvre l'interface de chat intégrée (voir 9.4.6).
- Pas de screenshots, vidéo, avis, ou sandbox au MVP.
- SEO : métadonnées dynamiques (Next.js generateMetadata), OG tags pour le partage social.

9.4.4 Publication d'Agents (Développeur)

- Page /dashboard/new-agent : formulaire en 3 étapes (Upload .agentjson → Métadonnées → Confirmation).
- Validation client du .agentjson (Zod) avec feedback instantané (erreurs affichées inline).
- Validation serveur basique : schéma JSON, vérification que les champs obligatoires sont présents, pas de clé API en dur dans le fichier.

- Métadonnées : nom, description courte, description longue (Markdown), catégorie, tags (max 5), modèle IA.
- Statut après soumission : « pending ». Review manuelle par admin (acceptable à l'échelle du MVP).
- Dashboard développeur basique : liste des agents publiés avec statut, bouton « Modifier » / « Nouvelle version ».

9.4.5 Panel Admin

- Page /admin (accès restreint par RLS + middleware) : liste des agents en attente (status=pending).
- Pour chaque agent : preview du .agentjson, métadonnées, boutons Approuver / Rejeter (avec champ motif).
- Notification email au développeur lors du changement de statut (via Resend).
- Statistiques basiques : nombre d'agents, nombre d'utilisateurs, nombre de développeurs.

9.4.6 Interface de Chat Web (Cœur du MVP)

L'interface de chat est la fonctionnalité la plus critique du MVP. C'est elle qui démontre la valeur d'AgentPlace : utiliser un agent IA directement depuis le navigateur.

- **Architecture côté client** : Le chat tourne entièrement dans le navigateur. L'utilisateur fournit sa clé API (stockée uniquement en mémoire, jamais persistée côté serveur). Les requêtes partent directement du navigateur vers l'API du modèle IA (Claude, GPT-4, etc.).
- **Première utilisation** : Modal demandant à l'utilisateur de saisir sa clé API pour le modèle requis. La clé est stockée en sessionStorage (disparaît à la fermeture de l'onglet) ou optionnellement en localStorage (persistée localement).
- **Streaming** : Les réponses sont streamées via SSE (Server-Sent Events) ou via l'API de streaming native du modèle (ex : stream: true pour l'API Anthropic).
- **Rendu Markdown** : Les réponses de l'agent sont rendues en Markdown (react-markdown) avec coloration syntaxique du code (rehype-highlight ou Prism).
- **System prompt** : Le system_prompt du .agentjson est injecté automatiquement. L'utilisateur ne le voit pas mais peut le consulter sur la fiche agent.
- **Tools / Function calls** : Pas de support des tools au stade MVP. Les agents sont des « chatbots » purs (system prompt + conversation). Le support des tools sera ajouté avec le moteur d'exécution desktop (Sprint 6).
- **Historique** : Les conversations sont stockées en localStorage (côté client uniquement). Pas de sync serveur au MVP.
- **Limitation mode Cloud** : Pour les agents en mode Cloud, les requêtes sont envoyées à l'endpoint du développeur via un proxy API Route Next.js (pour éviter les problèmes CORS). L'auth est gérée par un token temporaire.

Sécurité des clés API dans le MVP

Risque : les clés API sont exposées côté client (JavaScript). C'est un compromis acceptable pour le MVP car :

- L'utilisateur fournit volontairement SA clé (il en est responsable).
- Les clés ne sont jamais envoyées au serveur AgentPlace.
- La clé n'est visible que dans l'onglet de l'utilisateur (pas de persistance serveur).
- Dans le CDC complet, le problème est résolu par l'app desktop (clés dans le keychain OS natif, jamais exposées en JS).

9.5 Format .agentjson MVP (simplifié)

Le format MVP est un sous-ensemble du .agentjson v2. Les champs avancés (permissions, dependencies, sandbox_config, health_check, localization) sont optionnels et ignorés par le moteur MVP. Ils seront activés progressivement dans les sprints suivants.

Champ	Type	Requis MVP	Description
schema_version	string	Oui	Toujours « 2.0 »
name	string	Oui	Nom de l'agent
version	string	Oui	Version sémantique (ex : 1.0.0)
mode	enum	Oui	local cloud (hybrid ignoré au MVP)
model	string	Oui	Un seul modèle (string, pas array au MVP)
endpoint	string null	Si cloud	URL endpoint HTTPS du développeur
system_prompt	string	Oui	Instructions système (max 10 000 car.)
tools	array	Non	Ignoré au MVP (pas de support tools)
config	object	Non	temperature, max_tokens (valeurs par défaut si absent)
metadata	object	Oui	Icône URL, catégorie, tags[], description courte

Exemple minimaliste d'un .agentjson MVP :

```
{ "schema_version": "2.0", "name": "Correcteur Français", "version": "1.0.0",  
  "mode": "local", "model": "claude-sonnet-4-20250514",  
  "system_prompt": "Tu es un correcteur de français expert. Corrige les erreurs...",  
  "config": { "temperature": 0.3, "max_tokens": 2048 },  
  "metadata": { "category": "Productivité", "tags": ["français", "correction"],  
    "description": "Corrige et améliore vos textes en français." } }
```

9.6 Design et UX du MVP

- **Design system minimal** : Tailwind CSS + shadcn/ui (gratuit, composants accessibles). Pas de design custom au MVP. Les composants shadcn/ui sont utilisés tels quels avec une palette de couleurs personnalisée.
- **Palette MVP** : Fond blanc/gris clair, accent bleu foncé (#1D3557), accent rouge (#E63946) pour les CTAs. Mode sombre natif (prefers-color-scheme).

- **Responsive** : Mobile-first. Le catalogue et le chat doivent fonctionner correctement sur mobile même sans app native dédiée.
- **Pas de Figma au MVP** : Le design est fait directement en code (Tailwind + shadcn). Les composants shadcn/ui fournissent un niveau de qualité suffisant pour un MVP.
- **Accessibilité de base** : Navigation au clavier, labels ARIA sur les éléments interactifs, contraste suffisant (garanti par shadcn/ui).

9.7 Limites Acceptées et Risques du MVP

Limitation	Impact	Solution dans le CDC complet
Clés API exposées côté client	Risque sécurité faible (clé de l'utilisateur, son onglet)	App desktop avec keychain OS natif (Sprint 5-6)
Review manuelle des agents	Goulet d'étranglement si > 20 agents/semaine	Pipeline de validation automatisée (Sprint 3)
Pas de paiement	Pas de revenus, pas de monétisation développeur	Stripe Connect (Sprint 4)
Pas de cache Redis	Latence plus élevée si beaucoup de requêtes	Redis Upstash (Sprint 0 complet)
Pas de tools/function calls	Les agents sont limités au chat pur	Moteur d'exécution desktop (Sprint 6)
Pas de monitoring avancé	Détection d'erreurs limitée	Sentry + Better Stack (Sprint 0 complet)
500 MB DB max (Supabase Free)	Limite à ~5 000 agents environ	Supabase Pro à 25€/mois dès que nécessaire
Pas d'app native	Expérience mobile dégradée	React Native (Sprint 8), Tauri (Sprint 5-6)
Historique conversations local	Perdu si l'utilisateur change de navigateur	Sync serveur + app desktop (Sprint 5-6)

9.8 Critères de Succès du MVP

Les métriques du MVP sont volontairement modestes. L'objectif n'est pas la croissance mais la validation du concept.

Indicateur	Cible MVP (M+1)	Outil de mesure	Seuil Go pour CDC complet
Agents publiés	5+ agents approuvés	Query DB	> 3 agents de développeurs différents
Développeurs inscrits	10+ comptes dev	Supabase Auth	> 5 développeurs actifs (1+ agent)
Utilisateurs inscrits	50+	Supabase Auth	> 30 utilisateurs ayant utilisé un agent

Indicateur	Cible MVP (M+1)	Outil de mesure	Seuil Go pour CDC complet
Sessions de chat	100+	PostHog events	> 50 sessions de chat complètes
Feedbacks qualitatifs	10+ retours	Formulaire TypeForm / Google Forms	> 5 retours positifs sur le concept
Bugs critiques	0	Sentry (gratuit)	Aucun bug bloquant non résolu
Disponibilité	> 99%	Vercel status	Pas de downtime majeur (> 1h)

Critère Go/No-Go vers le CDC complet

GO si : au moins 3 développeurs externes ont publié un agent, au moins 30 utilisateurs ont utilisé le chat, et les retours qualitatifs confirment l'intérêt du concept.

NO-GO si : aucun développeur externe ne publie d'agent après 1 mois de promotion, ou les utilisateurs abandonnent systématiquement au moment de saisir leur clé API.

PIVOT si : les retours indiquent que le concept est bon mais que le modèle (clé API utilisateur) est un frein. Solution : basculer vers un modèle cloud-first où AgentPlace fournit les clés API (modèle avec crédits/abonnement).

9.9 Stratégie de Transition MVP → CDC Complet

Le code du MVP est conçu pour être réutilisé intégralement dans le produit complet. La transition se fait de manière incrémentale, sprint par sprint.

Composant MVP	Transition	Sprint CDC concerné
Next.js API Routes	Migration progressive vers Fastify (API séparée sur Fly.io). Les API Routes restent en place comme proxy pendant la migration.	Sprint 0 (complet)
Supabase Free	Upgrade vers Supabase Pro (25€/mois). Aucun changement de code, juste un changement de plan.	Sprint 0 (complet)
Vercel Hobby	Upgrade vers Vercel Pro (20€/mois) pour le custom domain et les preview deploys illimités.	Sprint 0 (complet)
Chat côté client	Le chat web est conservé tel quel. Le moteur d'exécution avancé (tools, function calls) est ajouté dans l'app desktop Tauri.	Sprint 5-6
Pas de paiement	Ajout de Stripe Connect. Les agents gratuits continuent de fonctionner, les agents payants deviennent disponibles.	Sprint 4
Review manuelle	Remplacée par le pipeline de validation automatisée. La review manuelle reste en fallback pour les cas limites.	Sprint 3
Modèle de données réduit	Ajout des tables manquantes via migrations Drizzle. Aucune table MVP n'est modifiée, seuls des champs et tables sont ajoutés.	Progressif (Sprint 2-9)

Composant MVP	Transition	Sprint CDC concerné
Pas de Redis	Ajout d'Upstash Redis pour le cache catalogue et le rate limiting.	Sprint 0 (complet)
Sentry Free	Upgrade vers Sentry Team (26€/mois) + ajout Better Stack pour les logs.	Sprint 0 (complet)

10. Découpage en Sprints MVP

Le MVP est découpé en 2 sprints de 2 semaines (4 semaines total). L'équipe de 2 développeurs travaille en parallèle avec une répartition frontend/backend. À l'issue du Sprint MVP-2, le produit est déployé en production et ouvert au public.

Position dans le planning global

Les Sprints MVP-1 et MVP-2 précèdent le Sprint 0 du CDC complet.

Si le MVP est validé (critères Go atteints), l'équipe enchaîne directement avec le Sprint 0 qui transforme l'infrastructure MVP en infrastructure production.

Planning total : 4 semaines MVP + 24 semaines CDC complet = 28 semaines (7 mois).

Sprint MVP-1 — Fondations & Catalogue | Semaines 1-2

Objectif : Environnement configuré, auth fonctionnelle, catalogue d'agents navigable, publication d'agents opérationnelle.

#	User Story	Prio	Pts	Assigné
M1-1	Initialisation projet Next.js 14 (App Router) + Tailwind CSS + shadcn/ui. Structure de dossiers, ESLint, Prettier, TypeScript strict. Déploiement initial sur Vercel (Hobby).	Must	3	A
M1-2	Configuration Supabase Free : création du projet (région EU), schéma DB MVP (tables users, agents, agent_versions), migrations Drizzle, RLS policies de base.	Must	5	B
M1-3	Auth : inscription/connexion email + GitHub OAuth via Supabase Auth. Pages /login, /signup, /forgot-password. Middleware de protection des routes. Session management.	Must	5	B
M1-4	Profils : page profil utilisateur (avatar, bio), page profil développeur (publique, liste agents). Sélection type de compte à l'inscription.	Must	5	A
M1-5	Landing page : hero section, barre de recherche, section « Comment ça marche », grille catégories, derniers agents, footer.	Must	5	A
M1-6	Catalogue : page /agents, cards agents, filtres (catégorie, modèle IA, recherche texte), pagination simple, query params URL.	Must	5	A
M1-7	API catalogue : endpoint de recherche full-text (ts_vector PostgreSQL), endpoint liste agents avec filtres, endpoint agent par slug.	Must	3	B

#	User Story	Prio	Pts	Assigné
M1-8	Publication d'agents : formulaire wizard 3 étapes (upload .agentjson + métadonnées + confirmation), validation Zod client + serveur, upload vers Supabase Storage.	Must	8	A + B
M1-9	Setup PostHog (analytics gratuit) + Sentry (error tracking gratuit). Events clés : signup, agent_created, agent_viewed.	Should	2	B

Sprint MVP-2 — Chat, Admin & Lancement | Semaines 3-4

Objectif : Interface de chat fonctionnelle, panel admin opérationnel, fiche agent complète, déploiement production.

#	User Story	Prio	Pts	Assigné
M2-1	Fiche agent détaillée : page /agents/[slug], description Markdown (react-markdown), infos techniques, bouton « Utiliser cet agent », SEO (generateMetadata, OG tags).	Must	5	A
M2-2	Interface de chat web : composant chat complet (input, messages, streaming), gestion de la clé API utilisateur (modal de saisie, stockage sessionStorage), appel direct aux APIs modèles IA depuis le navigateur.	Must	8	A
M2-3	Adaptateurs modèles IA côté client : fonctions d'appel pour Anthropic (Claude), OpenAI (GPT-4), Google (Gemini), Mistral. Gestion du streaming SSE. Interface commune.	Must	8	B
M2-4	Proxy API Route pour agents Cloud : endpoint Next.js API Route qui relaie les requêtes vers l'endpoint du développeur (contourne les problèmes CORS). Auth par token temporaire.	Must	3	B
M2-5	Panel admin : page /admin (accès restreint), liste agents pending, preview .agentjson, boutons approuver/rejeter, notification email (Resend).	Must	5	A + B
M2-6	Dashboard développeur basique : page /dashboard, liste des agents avec statut, compteurs simples (agents publiés, agents approuvés).	Must	3	A
M2-7	Historique conversations : stockage localStorage, liste des conversations passées par agent, possibilité de reprendre une conversation.	Should	3	A
M2-8	Pages légales minimales : mentions légales, CGU basiques, politique de confidentialité simplifiée. Conforme au minimum légal français.	Must	2	A
M2-9	Tests et déploiement final : smoke tests manuels sur les parcours critiques (inscription → publication → chat), correction des bugs critiques, déploiement production Vercel.	Must	3	A + B
M2-10	Seed data : création de 3-5 agents d'exemple (productivité, code, créatif) pour que la marketplace ne soit pas vide au lancement.	Must	2	A + B

10.1 Roadmap Complète (MVP + CDC)

Phase	Sprints	Durée	Livrable	Coût infra
Phase MVP	MVP-1 + MVP-2	4 sem.	Marketplace web fonctionnelle (agents gratuits + chat)	0€/mois
→ Validation	1-2 mois de test	4-8 sem.	Feedback utilisateurs, métriques Go/No-Go	0€/mois
Phase 0 — Setup	Sprint 0	2 sem.	Infra production (Redis, CDN, monitoring avancé)	~155€/mois
Phase 1 — Core Web	Sprints 1 à 4	8 sem.	Paiements, avis, analytics, catalogue avancé	~155€/mois
Phase 2 — Desktop	Sprints 5-6	4 sem.	App Tauri (moteur local, keychain, multi-modèles)	~155€/mois
Phase 3 — Feedback	Sprint 7	2 sem.	Avis, recommandations, analytics dev	~155€/mois
Phase 4 — Mobile	Sprint 8	2 sem.	App React Native iOS + Android	~165€/mois
Phase 5 — Multi-Agents	Sprint 9	2 sem.	Pipelines, Agent Sélecteur	~165€/mois
Phase 6 — Polish	Sprint 10	2 sem.	Mobile avancé, dette technique	~165€/mois
Phase 7 — Launch	Sprint 11	2 sem.	Sécurité, SEO, production complète	~200€/mois

Timeline totale avec MVP

MVP : 4 semaines de développement + 4-8 semaines de validation marché.

CDC complet : 24 semaines de développement.

Total : 32-36 semaines (8-9 mois) du premier commit au lancement complet.

Coût total d'infrastructure sur la période : ~0€ (MVP) puis ~155-200€/mois (CDC complet), soit environ 1 000€ sur l'ensemble du projet.

11. Glossaire

Terme	Définition
MVP (Minimum Viable Product)	Version minimale du produit contenant uniquement les fonctionnalités essentielles pour valider le concept auprès des premiers utilisateurs, avant de développer le produit complet.
.agentjson	Format de fichier JSON standardisé décrivant la configuration complète d'un agent IA (identité, modèle, permissions, métadonnées).
Agent local	Agent dont l'exécution se fait sur la machine de l'utilisateur. Nécessite une clé API personnelle pour le modèle IA.
Agent cloud	Agent dont l'exécution est gérée par le développeur via un endpoint API hébergé.
Pipeline	Workflow multi-agents où plusieurs agents sont chaînés. L'output de chaque agent alimente l'input du suivant.

Terme	Définition
Sandbox	Mode d'essai permettant de tester un agent payant gratuitement pendant un nombre limité d'interactions.
Agent Sélecteur	Meta-agent intégré qui analyse la requête utilisateur et recommande automatiquement le meilleur agent de la bibliothèque.
MCP (Model Context Protocol)	Protocole standardisé pour connecter des outils externes aux modèles IA (développé par Anthropic).
RLS (Row Level Security)	Mécanisme PostgreSQL/Supabase qui restreint l'accès aux lignes d'une table en fonction de l'utilisateur connecté.
SSE (Server-Sent Events)	Protocole de streaming unidirectionnel permettant au serveur d'envoyer des données en continu au client (utilisé pour le streaming des réponses IA).
pgvector	Extension PostgreSQL permettant de stocker et rechercher des embeddings vectoriels (utilisé pour les recommandations et la recherche sémantique).

Fin du Cahier des Charges

AgentPlace v2.0 — Mars 2026

Ce document est vivant et sera mis à jour à chaque rétrospective de sprint.